

Smali By bithowl: Chapter 3 Android Bytecode Fundamentals

("Every Android App Speaks a Language Your Phone Understands")

SMALI BY BITHOWL : CHAPTER 3

ANDROID BYTECODE FUNDAMENTALS

"Every Android App Speaks a Language Your Phone Understands"

1 THE STORY (HOOK)

ENGLISH → **你好**

Developers write apps in Java or Kotlin. Android doesn't understand either language. Before an app can run, the code must be translated into instructions that Android can execute. That translated language is DEX bytecode. And Smali is the readable version of that language.

JADX is the witness. Smali is the CCTV footage.

2 WHY BYTECODE EXISTS

Computers don't understand Java. They don't understand Kotlin. Processors only understand machine instructions.

Android converts code into an intermediate format called DEX (Dalvik Executable).

Benefits:

- ✓ Better optimization for mobile devices
- ✓ Smaller application size
- ✓ Faster loading
- ✓ Reduced memory usage
- ✓ Platform independence

3 MEET THE DALVIK VIRTUAL MACHINE (DVM)

Android's original runtime (2008). Designed specifically for mobile devices with limited resources.

Key Difference from JVM:

Register-based architecture (instead of stack-based)

More efficient execution on mobile hardware.

4 ART (ANDROID'S MODERN RUNTIME)

Introduced in Android 5.0 (Lollipop). Replaced Dalvik but still loads DEX files.

- ✓ AOT (Ahead-Of-Time) compilation
- ✓ JIT (Just-In-Time) compilation
- ✓ Profile-guided optimizations
- ✓ Better performance & security

DEX is still the core. Smali is still relevant.

5 WHAT IS A DEX FILE?

DEX stands for Dalvik Executable. Android's executable format.

Instead of hundreds of .class files, Kotlin eventually ends up into one optimized file.

You'll usually find:

classes.dex

Large apps may also contain (MultiDex):

classes2.dex classes3.dex classes4.dex

Each DEX file contains:

- Classes
- Fields
- Bytecode instructions
- Constants
- Metadata

In short, it's the compiled logic of your application.

6 UNDERSTANDING classes.dex

If an APK is a package, then classes.dex is its brain.

Everything you've written in Java/Kotlin eventually ends up here.

classes.dex → APKTool (Baksmali) → smali/

Example structure:

```
smali/
├── MainActivity.smali
├── LoginActivity.smali
├── ApiClient.smali
└── Utils.smali
```

Each Smali file represents a class originally compiled into the DEX file.

7 HOW ANDROID EXECUTES BYTECODE

User taps the app

Android loads the APK

Reads classes.dex

ART loads the required classes

Methods are compiled or interpreted

CPU executes machine instructions

At no point does Android read Java or Kotlin source code. The runtime only works with compiled bytecode.

8 THE ANDROID COMPILER PIPELINE

Java / Kotlin Source Code

Java Compiler

Kotlin Bytecode (.class)

DB / R8 Compiler

DEX (.dex)

Smali (Human-readable)

Android Runtime (ART)

Machine Code

Code is transformed multiple times. That's why decompilers can't perfectly recover the original source.

9 BUG HUNTER PERSPECTIVE

JADX shows a simple method. Looks secure.

```
if (isLoggedIn()) {
    openDashboard();
}
```

But after inspecting Smali, you discover additional logic that changes the actual execution path.

Bottom line:
The vulnerability isn't in the Java view. It's in the bytecode.

Experienced researchers use a 3-step workflow:

- 1 JADX → For readability
- 2 Smali → For accuracy
- 3 Frida → For runtime verification

10 WHERE SMALI HELPS IN REAL ASSESSMENTS

- Hidden Activities, Services, Broadcast Receivers
- SSL/TLS Certificate Pinning
- Root / Emulator Detection Logic
- Client-side Auth & Authorization Checks
- Deep Link Handling & Intent Processing
- Hardcoded Secrets & API Keys
- Input Validation Logic
- Business Logic Flaws
- Feature Flags & Debug Functionality
- Obfuscated Applications

11 WHY SECURITY RESEARCHERS USE SMALI

- Malware Analysis: Understand hidden behaviour, persistence and payloads.
- Bug Bounty Hunting: Verify vulnerabilities by analysing actual bytecode logic.
- Security Auditing: Review permissions, exported components, crypto, and more.
- Reverse Engineering: Understand application logic without source code.
- Patch Analysis: Compare versions and study how vulnerabilities were fixed.

12 PRACTICAL EXERCISE

Let's explore an APK!

Tools You Need

- APKTool
- JADX
- Sample APK

Step 1: Extract APK

```
$ apktool d app.apk
```

Step 2: Notice the structure

```
app/
├── AndroidManifest.xml
├── res/
├── assets/
├── lib/
├── smali/
└── smali_classes2/
    └── unknown/
```

Step 3: Compare Files

Original APK: classes.dex, classes2.dex, ...

Extracted Smali: smali/, smali_classes2/, ...

Step 4: Open & Compare

- ✓ Open the same Activity in JADX and Smali
- ✓ Compare methods, variables, control flow
- ✓ Which view is more accurate?

13 QUICK RECAP

- ✓ Android executes DEX bytecode, not Java/Kotlin source.
- ✓ Dalvik introduced DEX and a register-based execution model.
- ✓ ART replaced Dalvik but still loads DEX files from APKs.
- ✓ classes.dex contains the core logic; additional classesN.dex files support MultiDex.
- ✓ Smali is the human-readable representation of DEX.

14 FINAL THOUGHT

Most developers think in Java. Android thinks in bytecode. And every reverse engineer eventually realizes the same truth: If you want to understand what an application says, read the source code. If you want to understand what it does, follow the bytecode.

SOURCE CODE (What it says) → VS → **BYTECODE** (What it does) .dex

BOTTOM LINE

A decompiler tells you what an application probably does. Smali shows you what it actually does. Understand the bytecode. Understand the truth.

By - bithowl

When you tap an app icon, Android doesn't execute Java. It doesn't execute Kotlin either. Those languages are only for developers. Your phone speaks something entirely different.

A language built for speed.

A language built for efficiency.

A language made of **DEX bytecode**.

And if you want to truly understand Android security, you need to understand how that bytecode travels from your IDE to your phone's processor.

Because every vulnerability... Every authentication check... Every API call... Every security decision...

Eventually becomes bytecode. Let's follow that journey.

The Story (Hook)

Imagine you're traveling to another country.

You speak English. The locals don't.

Before your message reaches them, it must be translated into a language they understand. Android applications work the same way. Developers write apps in Java or Kotlin. But Android doesn't understand either language.

Before an app can run, the code must be translated into instructions that Android's runtime can execute. That translated language is **DEX bytecode**.

And Smali is simply the readable version of that language.

Understanding this translation process is one of the biggest steps toward becoming an Android reverse engineer.

Why Bytecode Exists

Computers don't understand Java. They don't understand Kotlin.

Processors only understand machine instructions.

Instead of converting Java directly into machine code, Android first converts it into an intermediate format called **Dalvik Executable (DEX)**.

This provides several advantages:

- Better optimization for mobile devices
- Smaller application size
- Faster loading
- Reduced memory usage
- Platform independence

DEX acts as the bridge between your source code and the Android Runtime.

Meet the Dalvik Virtual Machine (DVM)

When Android was first released in 2008, applications didn't run directly on the operating system.

Instead, every application executed inside the **Dalvik Virtual Machine (DVM)**.

Unlike the Java Virtual Machine (JVM), Dalvik was specifically designed for smartphones, where memory, battery, and CPU resources were limited.

One of its biggest differences was its **register based architecture**.

Instead of relying on a stack like the JVM, Dalvik used registers to perform operations more efficiently on mobile hardware.

This design became the foundation of Android bytecode.

ART (Android's Modern Runtime)

Starting with Android 5.0 (Lollipop), Google replaced Dalvik with the **Android Runtime (ART)**. Many beginners assume this made Smali obsolete. It didn't.

Here's why.

Applications are **still distributed as DEX files**. ART simply executes them differently.

Instead of interpreting every instruction at runtime like Dalvik, ART compiles much of the bytecode into native machine code ahead of time (AOT) and can also use just-in-time (JIT) compilation and profile-guided optimizations.

For reverse engineers, the important point is this: The APK still contains DEX files. And DEX files still become Smali.

Whether the device uses Dalvik or ART, Smali remains the most faithful representation of the application's packaged logic.

What Is a DEX File?

DEX stands for **Dalvik Executable**. Think of it as Android's executable format.

Instead of storing hundreds of individual .class files like Java, Android combines them into one optimized file.

Usually you'll find:

classes.dex

Large applications may also contain:

classes2.dex

classes3.dex

classes4.dex

This is known as **MultiDex**.

Each DEX file contains:

- Classes

- Methods
- Fields
- Constants
- Bytecode instructions
- Metadata

In short, it's the compiled logic of your application.

Understanding classes.dex

If an APK is a package, then classes.dex is its brain.

Everything you've written in Java or Kotlin eventually ends up here.

When APKTool disassembles an APK, it converts the contents of classes.dex into readable Smali files.

For example:

classes.dex

|



smali/

|— MainActivity.smali

|— LoginActivity.smali

|— ApiClient.smali

|— Utils.smali

Each Smali file represents a class originally compiled into the DEX file.

How Android Executes Bytecode

Let's walk through what happens when you launch an application.

User taps the app

|



Android loads the APK

|



Reads classes.dex

|



ART loads the required classes



Methods are compiled or interpreted



CPU executes machine instructions

Notice something important. At no point does Android read Java or Kotlin source code. The runtime only works with compiled bytecode.

That's why reverse engineers focus on DEX and Smali—not the original programming language.

The Android Compiler Pipeline

Every Android application follows the same journey.

Java / Kotlin



Java Compiler



Java Bytecode (.class)



D8 / R8 Compiler



DEX (.dex)



Smali (Human-readable representation)



Android Runtime (ART)

|



Machine Code

Understanding this pipeline explains why decompilers can never perfectly recover the original source code. The original Java has already been transformed multiple times before reaching the device.

Bug Hunter Perspective

Many beginners jump straight into JADX. Experienced researchers don't stop there. They understand where the decompiled Java came from. Suppose you're auditing an application that implements SSL pinning.

JADX shows a simple method. Looks secure.

But after inspecting the corresponding Smali, you notice additional method calls, compiler optimizations, or logic that changes the actual execution path. The vulnerability isn't in the Java view.

It's in the bytecode.

This is why Android bug hunters frequently switch between:

- JADX for readability
- Smali for accuracy
- Frida for runtime verification

Understanding the compilation pipeline helps you know **which representation to trust** during different stages of an assessment.

Practical Exercise

Choose any APK and extract it using APKTool: **apktool d app.apk**

Locate the generated directories:

smali/

smali_classes2/

Now compare them with the original APK.

You'll also notice:

classes.dex

classes2.dex

Open one Smali class and one Decompiled Java class side by side.

Ask yourself:

- Can I identify the same method?
- How were variable names reconstructed?

- Do both views show the same control flow?
- Which version appears closer to the actual bytecode?

Quick Recap

- Android devices execute **DEX bytecode**, not Java or Kotlin source code.
- Dalvik introduced the DEX format and a register-based execution model.
- ART replaced Dalvik as the runtime but still loads DEX files from APKs.
- `classes.dex` contains the application's compiled logic, while additional files like `classes2.dex` support MultiDex apps.
- Smali is a human-readable representation of DEX bytecode, making it a valuable tool for reverse engineering.
- Understanding the compilation pipeline helps security researchers interpret decompiled code more accurately.

Final Thought

Most developers think in Java. Android thinks in bytecode.

And every reverse engineer eventually realizes the same truth: If you want to understand what an application **says**, read the source code. If you want to understand what it **does**, follow the bytecode.

By - bithowl